Programming with Generative AI

Viraj Kumar
Kotak-IISc AI-ML Centre

Lecture 14:

- Algorithm-centric vs Data-centric solutions
- Demystifying Python lists



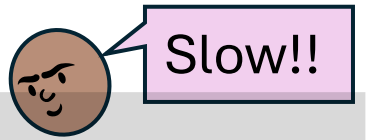
A CSR Initiative by



NPTEL

Recall: to_ind_lower

- PythonTutor



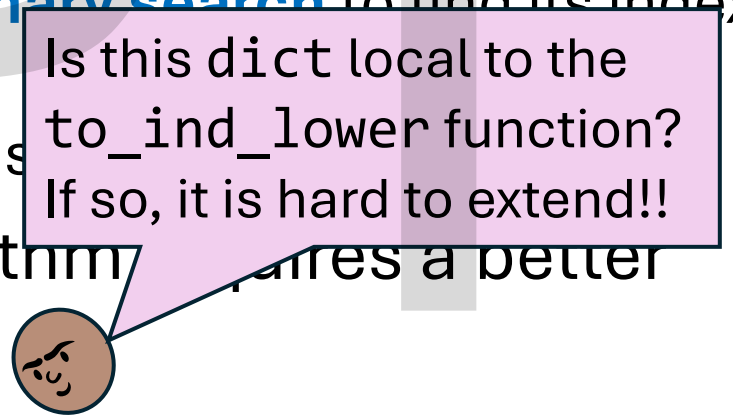

```
if (strcmp(word_lower, "color") == 0) {
  return "colour";
}
if (strcmp(word_lower, "favorite") == 0) {
  return "favourite";
}
if (strcmp(word_lower, "analyze") == 0) {
  return "analyse";
}
return word_lower;
```

NPTEL

Algorithm-centric improvements

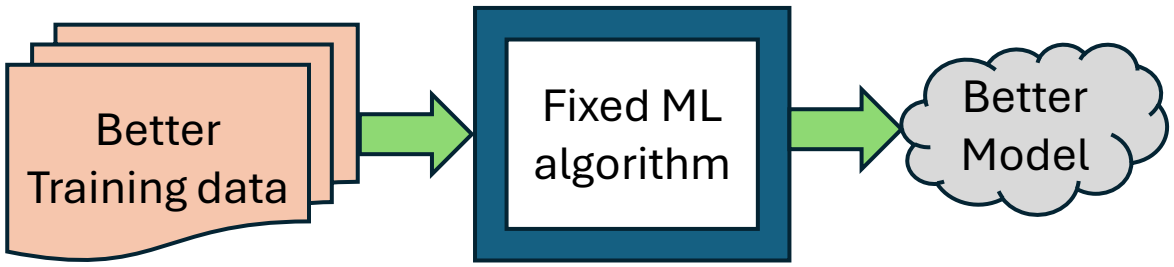
- Instead of a linear `if-else if- ... -else` search, maintain two "parallel" arrays:
  - `us`: Sorted lexicographically
  - Given a word, use **binary search** to find its index `i` in the array `us`
  - `ind[i]` is the Indian s
- Often, a better algorithm requires a better data-structure
- (*key*, *value*) pairs! `to_ind: dict[str, str]`
  - `to_ind[word]` is the Indian spelling of `word`

	0	1	2	…
us	analyze	color	favorite	…
ind	analyse	colour	favourite	…



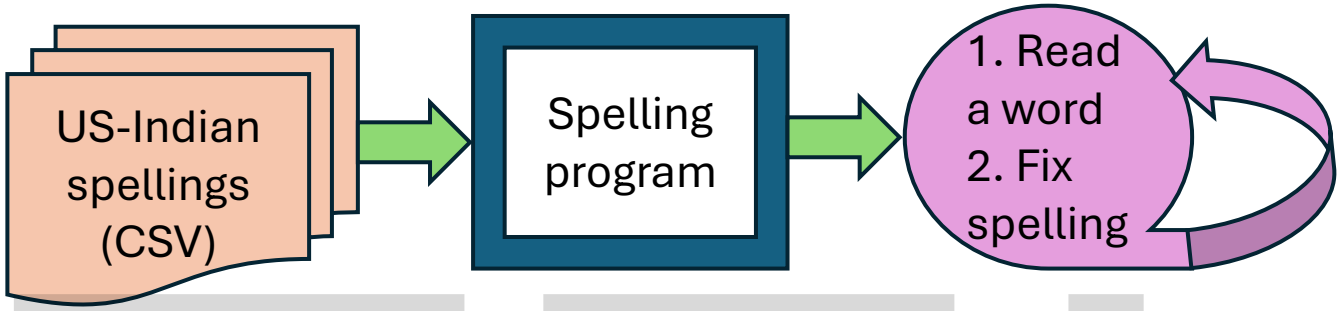
Data-centric improvements

- Store the (*key*, *value*) pairs in a **file**
  - E.g., a CSV file
- When the program starts, initialize the `dict` by reading (*key*, *value*) pairs from the file
- This **fixed** program can be improved by improving the **data** in the file
- Machine Learning:



	A	B
1	US spelling	Indian spelling
2	color	colour
3	analyze	analyse
4	favorite	favourite
5	…	…

C or Python?



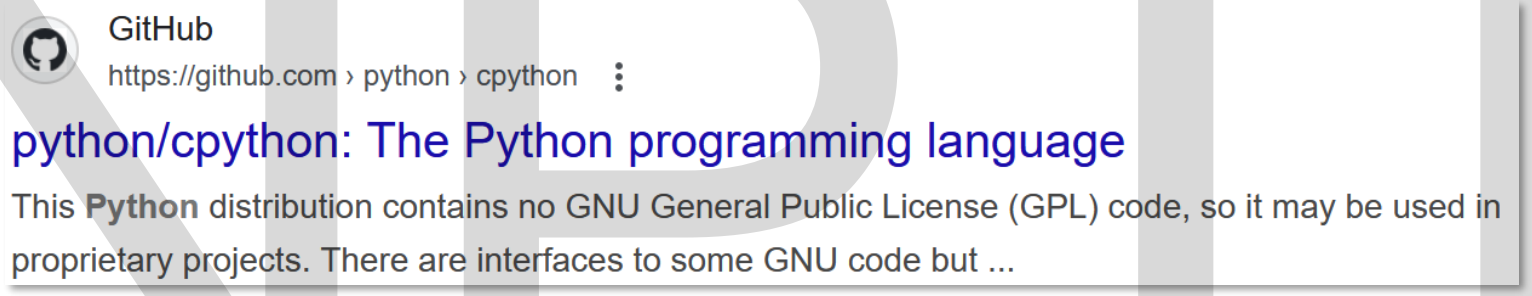
- Suggested approach:

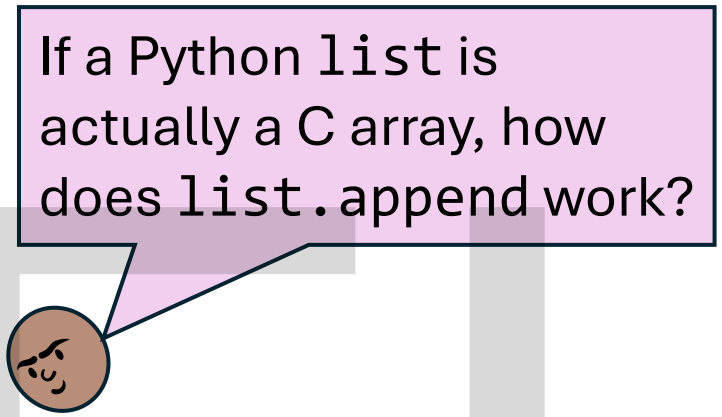
1. Implement the program in Python
2. Use a **profiler** to identify the **hot** parts of the code
   - *Example*: See Emery Berger's talk on Scalene
3. Translate *only* the hot parts into C
   - E.g., a function
4. Call that function from Python
   - Simple example
   - Official documentation

NPTEL

Demystifying Python lists

- One of the most popular implementations of Python is CPython, which is written in C!





- We will use Gen AI to develop a simplified `list[int]` implementation
  - `empty_list`
  - `len`
  - `index`
  - `append`

Summary

- Two important ways of improving: **algorithm-centric** and **data-centric**
- To improve (algorithmic) performance:
  - Start by writing a simple and correct implementation (e.g., in Python)
  - Profile the code to identify the hot parts
  - Make algorithm-centric improvements
  - Then, if necessary, translate your fast algorithm (e.g., from Python to C)
- There are brilliant gems hidden in the CPython implementation… start exploring!

NPTEL